

Chapter 8

The Relational Algebra and Relational Calculus

1

Chapter Outline

- Relational Algebra
 - Unary Relational Operations
 - Relational Algebra Operations from Set Theory
 - Binary Relational Operations
 - Additional Relational Operations
 - Examples of Queries in Relational Algebra

2

Relational Algebra Overview

- Relational algebra is the basic set of operations for the relational model.
- These operations enable a user to specify basic retrieval requests (or queries).
- **The result of each relational algebra operation on one or more input relations is a new relation.**
 - Each relation is a *set of tuples*, so cannot include duplicate tuples.
- A sequence of relational algebra operations forms a **relational algebra expression**.
 - Each relational algebra expression contains a operation or a sequence of operations.

3

Relational Algebra Overview (cont'd)

- Relational Algebra consists of several groups of operations.
 - Unary Relational Operations
 - SELECT (symbol: σ (sigma))
 - PROJECT (symbol: π (pi))
 - RENAME (symbol: ρ (rho))
 - Relational Operations from Set Theory
 - UNION ($\cup$), INTERSECTION ($\cap$), DIFFERENCE (or MINUS) ($-$)
 - CARTESIAN PRODUCT ($\times$)
 - Binary Relational Operations
 - JOIN (several variations of JOIN exist)
 - DIVISION
 - Additional Relational Operations
 - OUTER JOINS, OUTER UNION
 - AGGREGATE FUNCTIONS computing summary of information: SUM, COUNT, AVERAGE, MINIMUM, MAXIMUM

4

COMPANY Relational Database

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

5

SELECT Operation

- The SELECT operation (denoted by σ (sigma)) is used to select a subset of the tuples from a relation based on a selection condition.
 - The selection condition acts as a filter.
 - Only those tuples that satisfy the selection condition are included in the result (i.e., output relation).
 - Tuples that do not satisfy the selection condition are discarded (i.e., filtered out).
- In general, the select operation is denoted by $\sigma_{\langle \text{selection condition} \rangle}(R)$ where
 - the symbol σ (sigma) is used to denote the select operator.
 - the selection condition is a Boolean expression specified on the attributes of relation R.

6

SELECT Operation (cont'd)

- Each simple SELECT condition is in the form of $A_i \theta A_j$ or $A_i \theta \text{constant_value}$, where A_i and A_j are attributes of the operand relation and θ is one of the comparison operators $\{ =, \neq, >, \geq, <, \leq \}$.
- SELECT conditions can be combined by using AND or OR:
 - condition1 AND condition2
 - condition1 OR condition2
- (NOT condition1) is True if condition1 is False.
- Examples:
 - Select the EMPLOYEE tuples whose department number is 4:
 $\sigma_{\text{Dno} = 4}(\text{EMPLOYEE})$
 - Select the employee tuples whose salary is greater than \$30,000:
 $\sigma_{\text{Salary} > 30000}(\text{EMPLOYEE})$
 - Select the EMPLOYEE tuples whose department number is 4 and salary is greater than \$30,000: $\sigma_{\text{Dno} = 4 \text{ AND } \text{Salary} > 30000}(\text{EMPLOYEE})$
 - Select the EMPLOYEE tuples whose department number is 4 or salary is greater than \$30,000: $\sigma_{\text{Dno} = 4 \text{ OR } \text{Salary} > 30000}(\text{EMPLOYEE})$

7

SELECT Operation Properties

- The SELECT operation $\sigma_{\langle \text{selection condition} \rangle}(R)$ produces a relation, say S, that has the same schema (i.e., same attributes) as R.
- SELECT σ is commutative:

$$\sigma_{\langle \text{condition1} \rangle}(\sigma_{\langle \text{condition2} \rangle}(R)) = \sigma_{\langle \text{condition2} \rangle}(\sigma_{\langle \text{condition1} \rangle}(R))$$
- A sequence of SELECT operations may be applied in any order:

$$\sigma_{\langle \text{cond1} \rangle}(\sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond3} \rangle}(R))) = \sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond3} \rangle}(\sigma_{\langle \text{cond1} \rangle}(R)))$$
- A sequence of SELECT operations can be replaced by a single SELECT operation with a conjunction (i.e., logical ANDs) of all the conditions:

$$\sigma_{\langle \text{cond1} \rangle}(\sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond3} \rangle}(R))) = \sigma_{\langle \text{cond1} \rangle \text{ AND } \langle \text{cond2} \rangle \text{ AND } \langle \text{cond3} \rangle}(R)$$

8

PROJECT Operation

- PROJECT operation is denoted by π (pi)
- This operation keeps certain columns (i.e., attributes) from a relation and discards the other columns.
- The general form of the PROJECT operation is:
$$\pi_{\langle \text{attribute list} \rangle}(R)$$
 - $\langle \text{attribute list} \rangle$ is the desired list of attributes from relation R.
- **The PROJECT operation removes any *duplicate tuples* in the result.**
 - This is because the result of the PROJECT operation is also a relation, which is a *set of tuples*.
 - PROJECT and UNION may produce duplicate tuples, and they must be removed in their result relations.
- Example: To list each employee's first and last names and salary, the following PROJECT operation is used:

$\pi_{\text{Lname, Fname, Salary}}(\text{EMPLOYEE})$

9

PROJECT Operation Properties

- The number of tuples in the result of projection $\pi_{\langle \text{attribute list} \rangle}(R)$ is always less than or equal to the number of tuples in R, because duplicates are removed.
 - If the list of attributes includes a *key* of R, then there will be no duplicates in the result relation, so that the number of tuples in the result of PROJECT is equal to the number of tuples in R.
- PROJECT is *not* commutative:
$$\pi_{\langle \text{attribute list1} \rangle}(\pi_{\langle \text{attribute list2} \rangle}(R)) = \pi_{\langle \text{attribute list1} \rangle}(R)$$
as long as $\langle \text{attribute list2} \rangle$ contains all the attributes in $\langle \text{attribute list1} \rangle$.

10

Examples of SELECT and PROJECT Operations

Figure 8.1 Results of SELECT and PROJECT operations.

- (a) $\sigma_{(Dno=4 \text{ AND } Salary > 25000) \text{ OR } (Dno=5 \text{ AND } Salary > 30000)}(EMPLOYEE)$.
 (b) $\pi_{Lname, Fname, Salary}(EMPLOYEE)$. (c) $\pi_{Sex, Salary}(EMPLOYEE)$.

(a)

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5

(b)

Lname	Fname	Salary
Smith	John	30000
Wong	Franklin	40000
Zelaya	Alicia	25000
Wallace	Jennifer	43000
Narayan	Ramesh	38000
English	Joyce	25000
Jabbar	Ahmad	25000
Borg	James	55000

(c)

Sex	Salary
M	30000
M	40000
F	25000
F	43000
M	38000
M	25000
M	55000

Relational Algebra Expressions

- We may want to apply several relational algebra operations, one after the other.
 - We can write the operations **as a single relational algebra expression** by nesting the operations, or
 - We can apply one operation (or more) at a time and create **intermediate result relations**.
 - In the latter case, we must give names to the relations that hold the intermediate results.

Single Expression vs. a Sequence of Relational Operations (Example)

- To retrieve the first name, last name, and salary of all employees who work in department number 5, we apply a SELECT operation followed by a PROJECT operation.
 - We can write a single relational algebra expression as follows:

$$\text{RESULT} \leftarrow \pi_{\text{Fname, Lname, Salary}} (\sigma_{\text{Dno}=5} \text{EMPLOYEE})$$

- Or we can explicitly show the sequence of operations, giving a name to each intermediate result relation:

$$\text{DEP5_EMPS} \leftarrow \sigma_{\text{Dno}=5} (\text{EMPLOYEE})$$

$$\text{RESULT} \leftarrow \pi_{\text{Fname, Lname, Salary}} (\text{DEP5_EMPS})$$

13

RENAME Operations

- The RENAME operation is denoted by ρ (rho).
- In some cases, we may want to rename the attributes of a relation or the relation name or both.
 - To use more informative or short names in the subsequent relational algebra operations.
 - To unify the different attribute names in two relations, which is necessary in some cases (see the **Natural JOIN**, **DIVISION**, and **OUTER UNION** operations described later).
 - Rename operation doesn't change the name of a relation or the names of attributes in the real DB. They are renamed only for the subsequent relational algebra operations (in the query).

14

RENAME Operations (cont'd)

- The general RENAME operator ρ can be expressed by any of the following forms:
 - $\rho_{S(B_1, B_2, \dots, B_n)}(R)$ changes both:
 - the relation name to S, and
 - the attribute names to B1, B1, ..., Bn.
 - $\rho_S(R)$ changes:
 - only the relation name to S.
 - $\rho_{(B_1, B_2, \dots, B_n)}(R)$ changes:
 - only the attribute names to B1, B1, ..., Bn.

15

RENAME Operation (cont'd)

- For convenience, we also use a *shorthand notation* for renaming attributes in an intermediate relation:
 - If we write
$$R \leftarrow \pi_{Fname, Lname, Salary}(EMPLOYEE)$$
R will have the same 3 attribute names (Fname, Lname, Salary) as EMPLOYEE.
 - If we write
$$R(First_name, Last_name, Salary) \leftarrow \pi_{Fname, Lname, Salary}(EMPLOYEE)$$
then Fname, Lname, and Salary are renamed to First_name, Last_name, and Salary in R.
 - If we write
$$EMP \leftarrow EMPLOYEE$$
then EMPLOYEE relation is renamed to EMP.

16

Example of applying multiple operations and RENAME

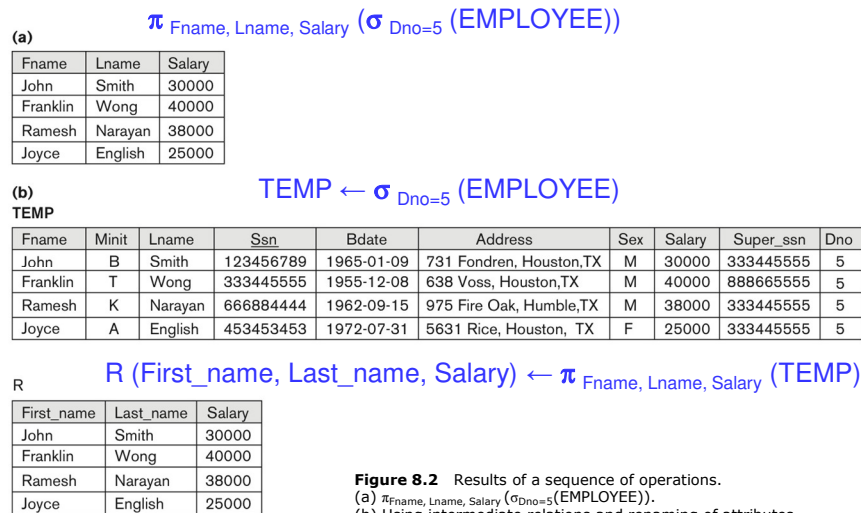


Figure 8.2 Results of a sequence of operations.
 (a) $\pi_{Fname, Lname, Salary} (\sigma_{Dno=5}(EMPLOYEE))$.
 (b) Using intermediate relations and renaming of attributes.

17

UNION Operation

- UNION is denoted by $\cup$.
- The result of $(R \cup S)$ is a relation that includes all the tuples that are either in R or S or in both relations.
- **Duplicate tuples are removed in the result of UNION** because the result of UNION is also a relation (i.e., a set of tuples).
- The two operand relations R and S must be “**type compatible**” (aka “**UNION compatible**”):
 - R and S must have same number of attributes.
 - Each pair of corresponding attributes must have the same domain (but they may have different names).
- The result relation of $(R1 \cup R2)$ has the same attribute names as the *first* operand relation R1 (by convention).

18

UNION Operation (cont'd)

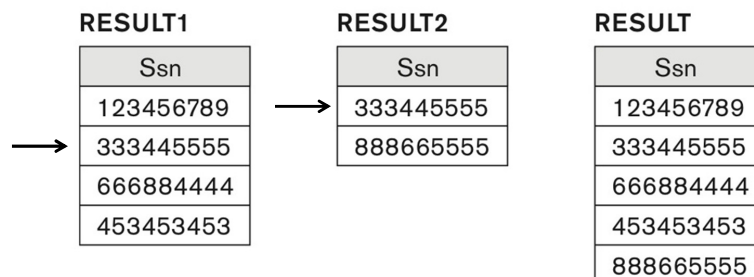
- Query: Retrieve the social security numbers of all the employees who either work in department 5 or directly supervise an employee who works in department 5.

```
DEP5_EMPS  $\leftarrow$   $\sigma_{Dno=5}$  (EMPLOYEE)  
RESULT1  $\leftarrow$   $\pi_{Ssn}$ (DEP5_EMPS)  
RESULT2(Ssn)  $\leftarrow$   $\pi_{Super\_ssn}$ (DEP5_EMPS)  
RESULT  $\leftarrow$  RESULT1  $\cup$  RESULT2
```

19

Example a UNION Operation

Figure 8.3 Result of the UNION operation $RESULT \leftarrow RESULT1 \cup RESULT2$.



20

INTERSECTION Operation

- INTERSECTION is denoted by $\cap$.
- The result of $(R \cap S)$ is a relation that includes all the tuples that are in both R and S.
 - Each common tuple should be included in the result of $(R \cap S)$ only once.
- The two operand relations R and S must be type compatible.
- The result relation of $(R1 \cap R2)$ has the same attribute names as the *first* operand relation R1 (by convention).

21

DIFFERENCE Operation

- DIFFERENCE (also called MINUS) is denoted by $-$.
- The result of $(R - S)$ is a relation that includes all the tuples that are in R but not in S.
 - The result of $(R - S)$ can be obtained by removing the common tuples from R.
- The two operand relations R and S must be type compatible.
- In general, $R - S \neq S - R$.
- The result relation of $(R1 - R2)$ has the same attribute names as the *first* operand relation R1 (by convention).

22

Examples of UNION, INTERSECTION, and DIFFERENCE

Figure 8.4 The set operations UNION, INTERSECTION, and MINUS.

(a) Two union-compatible relations.

(b) $\text{STUDENT} \cup \text{INSTRUCTOR}$.

(c) $\text{STUDENT} \cap \text{INSTRUCTOR}$.

(d) $\text{STUDENT} - \text{INSTRUCTOR}$.

(e) $\text{INSTRUCTOR} - \text{STUDENT}$.

(a) STUDENT

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

INSTRUCTOR

Fname	Lname
John	Smith
Ricardo	Browne
Susan	Yao
Francis	Johnson
Ramesh	Shah

(b)

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert
John	Smith
Ricardo	Browne
Francis	Johnson

(c)

Fn	Ln
Susan	Yao
Ramesh	Shah

(d)

Fn	Ln
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

(e)

Fname	Lname
John	Smith
Ricardo	Browne
Francis	Johnson

23

Some Properties of UNION, INTERSECTION, and DIFFERENCE

- Notice that both UNION and INTERSECTION are *commutative* operations:
 $R \cup S = S \cup R$
 $R \cap S = S \cap R$
- Both UNION and INTERSECTION are *associative* operations:
 $R \cup (S \cup T) = (R \cup S) \cup T$
 $(R \cap S) \cap T = R \cap (S \cap T)$
- The DIFFERENCE (or MINUS) operation is *not commutative*; that is, in general
 $R - S \neq S - R$

24

CARTESIAN PRODUCT Operation

- This operation is used to combine the tuples from two relations, and denoted by $\times$.
 $R(A_1, A_2, \dots, A_n) \times S(B_1, B_2, \dots, B_m)$
- The result is a relation, say Q, with $(n + m)$ attributes:
 $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$, in that order.
- The result relation has one tuple for each combination of tuples — one from R and one from S.
 - Hence, if R has n_R tuples and S has n_S tuples, then $(R \times S)$ will have $n_R * n_S$ tuples.
- The two operand relations don't need to be type compatible.
- CARTESIAN PRODUCT is also known as **CROSS PRODUCT**.

25

CARTESIAN PRODUCT Operation (cont'd)

$FEMALE_EMPS \leftarrow \sigma_{Sex = 'F'} (EMPLOYEE)$

FEMALE_EMPS

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5

EMPNames

Fname	Lname	Ssn
Alicia	Zelaya	999887777
Jennifer	Wallace	987654321
Joyce	English	453453453

$EMPNames \leftarrow \pi_{Fname, Lname, Ssn} (FEMALE_EMPS)$

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

26

CARTESIAN PRODUCT Operation (cont'd)

$EMP_DEPENDENTS \leftarrow EMPNAMES \times DEPENDENT$

EMP_DEPENDENTS

Fname	Lname	Ssn	Essn	Dependent_name	Sex	Bdate	...
Alicia	Zelaya	999887777	333445555	Alice	F	1986-04-05	...
Alicia	Zelaya	999887777	333445555	Theodore	M	1983-10-25	...
Alicia	Zelaya	999887777	333445555	Joy	F	1958-05-03	...
Alicia	Zelaya	999887777	987654321	Abner	M	1942-02-28	...
Alicia	Zelaya	999887777	123456789	Michael	M	1988-01-04	...
Alicia	Zelaya	999887777	123456789	Alice	F	1988-12-30	...
Alicia	Zelaya	999887777	123456789	Elizabeth	F	1967-05-05	...
Jennifer	Wallace	987654321	333445555	Alice	F	1986-04-05	...
Jennifer	Wallace	987654321	333445555	Theodore	M	1983-10-25	...
Jennifer	Wallace	987654321	333445555	Joy	F	1958-05-03	...
Jennifer	Wallace	987654321	987654321	Abner	M	1942-02-28	...
Jennifer	Wallace	987654321	123456789	Michael	M	1988-01-04	...
Jennifer	Wallace	987654321	123456789	Alice	F	1988-12-30	...
Jennifer	Wallace	987654321	123456789	Elizabeth	F	1967-05-05	...
Joyce	English	453453453	333445555	Alice	F	1986-04-05	...
Joyce	English	453453453	333445555	Theodore	M	1983-10-25	...
Joyce	English	453453453	333445555	Joy	F	1958-05-03	...
Joyce	English	453453453	987654321	Abner	M	1942-02-28	...
Joyce	English	453453453	123456789	Michael	M	1988-01-04	...
Joyce	English	453453453	123456789	Alice	F	1988-12-30	...
Joyce	English	453453453	123456789	Elizabeth	F	1967-05-05	...



CARTESIAN PRODUCT (cont'd)

- Generally, **CARTESIAN PRODUCT** is not a meaningful operation.
 - It can become meaningful when it is followed by another operation, such as **SELECT**.
- Example (not meaningful):

$EMP_DEPENDENTS \leftarrow EMPLOYEE \times DEPENDENT$

EMP_DEPENDENTS will contain every combination of an EMPLOYEE tuple and a DEPENDENT tuple regardless they are actually related or not.
- To keep only the combinations where the DEPENDENT is related to the EMPLOYEE, we add a **SELECT** operation as follows:

$ACTUAL_EMP_DEPENDENTS \leftarrow \sigma_{Ssn=Essn}(EMP_DEPENDENTS)$

28

JOIN Operations

- JOIN operation is denoted by $\bowtie$.
- The sequence of CARTESIAN PRODECT followed by SELECT can be used to identify and select related tuples from two relations. A JOIN operation combines this sequence into a single operation.
- Join operation is very important for any relational database with more than one relation, because it *allows us to combine related tuples from different relations*.
- The general form of a join operation on two relations $R(A_1, A_2, \dots, A_n)$ and $S(B_1, B_2, \dots, B_m)$ is:

$$R \bowtie_{\langle \text{join condition} \rangle} S$$

The join condition is in the form of $A_i \theta B_j$, where A_i and B_j are attributes of R and S , respectively, and θ is one of the comparison operators $\{ =, \neq, >, \geq, <, \leq \}$.

29

JOIN Operations (cont'd)

- Example: Suppose that we want to retrieve the information (such as name) of the manager of each department.
 - To get the manager's information, we need to combine each DEPARTMENT tuple with the EMPLOYEE tuple whose Ssn value matches the Mgr_ssn value in the department tuple.
 - We do this by using the following join operation.

$\text{DEPT_MGR} \leftarrow \text{DEPARTMENT} \bowtie_{\text{Mgr_ssn} = \text{Ssn}} \text{EMPLOYEE}$

- $\text{Mgr_ssn} = \text{Ssn}$ is the join condition.
- The join condition can also be specified as:
 $\text{DEPARTMENT.Mgr_Ssn} = \text{EMPLOYEE.Ssn}$
- This join is equivalent to
 $\sigma_{\text{Mgr_ssn} = \text{Ssn}} (\text{DEPARTMENT} \times \text{EMPLOYEE})$

30

Example of a JOIN Operation

DEPT_MGR $\leftarrow$ DEPARTMENT $\bowtie_{\text{Mgr_ssn} = \text{Ssn}}$ EMPLOYEE

DEPT_MGR

Dname	Dnumber	Mgr_ssn	...	Fname	Minit	Lname	Ssn	...
Research	5	333445555	...	Franklin	T	Wong	333445555	...
Administration	4	987654321	...	Jennifer	S	Wallace	987654321	...
Headquarters	1	888665555	...	James	E	Borg	888665555	...

31

Some Properties of JOIN

- Consider the following JOIN operation:
 $R(A_1, A_2, \dots, A_n) \bowtie_{R.A_i=S.B_j} S(B_1, B_2, \dots, B_m)$
 - Result is a relation, say Q, with $(n + m)$ attributes:
 - $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$, in that order.
 - The result relation has one tuple for each combination of tuples — t_R from R and t_S from S — only if they satisfy the join condition; that is, $t_R[A_i] = t_S[B_j]$.
 - Only related tuples of R and S (based on the join condition) will appear in the result.
 - Hence, if R has n_R tuples and S has n_S tuples, then the join result will generally have less than $n_R * n_S$ tuples.

32

Some Properties of JOIN (cont'd)

- A JOIN operation with a general join condition $R.A_i \theta S.B_j$ is called a **theta-join**, where θ is one of the comparison operators $\{ =, \neq, >, \geq, <, \leq \}$.
- Multiple join conditions between two relations can be combined by using **AND**. For example:

$R(A_1, A_2, \dots, A_n) \bowtie_{R.A_1=S.B_2 \text{ AND } R.A_2=S.B_4} S(B_1, B_2, \dots, B_m)$

$EMPLOYEE \bowtie_{Ssn=Essn \text{ AND } Sex=Sex} DEPENDENT$

33

EQUIJOIN Operation

- Among the comparison operators $\{ =, \neq, >, \geq, <, \leq \}$ that can be used in join conditions, $=$ is used most frequently.
- Such a join, where only $=$ is used as the comparison operator (in the join condition), is called an **EQUIJOIN**.
 - In the result of an EQUIJOIN, each pair of join attributes (whose names need not be identical) have identical values in every tuple.
 - In other words, **two columns corresponding to a pair of join attributes are identical in the result of EQUIJOIN**.

34

NATURAL JOIN Operation

- NATURAL JOIN denoted by * was created to get rid of the column corresponding to the second join attribute in each EQUIJOIN condition.
 - because one of the two identical columns, corresponding to a pair of join attributes, is redundant in the result relation.
- The standard definition of natural join requires that the two join attributes *have the same name* in both relations.

35

NATURAL JOIN (cont'd)

- Example 1: To apply a natural join on the Dnumber attributes of DEPARTMENT and DEPT_LOCATIONS, it is sufficient to write:
$$\text{DEPT_LOCS} \leftarrow \text{DEPARTMENT} * \text{DEPT_LOCATIONS}$$
 - Dnumber is only attribute having the same name in both relations.
 - An implicit join condition is created based on this attribute:
$$\text{DEPARTMENT.Dnumber} = \text{DEPT_LOCATIONS.Dnumber}$$
 - The result relation includes only the first Dnumber attribute.
- Example 2: $Q \leftarrow R(A,B,C,D) * S(C,D,E)$
 - The implicit join condition includes the equijoin between every *pair* of attributes with the same name, and they are ANDed together:
$$R.C=S.C \text{ AND } R.D=S.D$$
 - The result relation keeps only one join attribute from each pair:
$$Q(A,B,C,D,E)$$

36

Example of NATURAL JOIN Operation

$DEPT (Dname, Dnum, Mgr_ssn, Mgr_start_date) \leftarrow DEPARTMENT$

(a) $PROJ_DEPT \leftarrow PROJECT * DEPT$

PROJ_DEPT

Pname	Pnumber	Plocation	Dnum	Dname	Mgr_ssn	Mgr_start_date
ProductX	1	Bellaire	5	Research	333445555	1988-05-22
ProductY	2	Sugarland	5	Research	333445555	1988-05-22
ProductZ	3	Houston	5	Research	333445555	1988-05-22
Computerization	10	Stafford	4	Administration	987654321	1995-01-01
Reorganization	20	Houston	1	Headquarters	888665555	1981-06-19
Newbenefits	30	Stafford	4	Administration	987654321	1995-01-01

(b) $DEPT_LOCS \leftarrow DEPARTMENT * DEPT_LOCATIONS$

DEPT_LOCS

Dname	Dnumber	Mgr_ssn	Mgr_start_date	Location
Headquarters	1	888665555	1981-06-19	Houston
Administration	4	987654321	1995-01-01	Stafford
Research	5	333445555	1988-05-22	Bellaire
Research	5	333445555	1988-05-22	Sugarland
Research	5	333445555	1988-05-22	Houston

37

NATURAL JOIN Operation (cont'd)

- In the natural join, if two join attributes don't have the same name in both relations, the join attributes should be listed:

$Q \leftarrow R *_{(<\text{list of join attributes of } R>, <\text{list of join attributes of } S>)} S$

For example, $R *_{A, C} S$ if the join condition is $R.A=S.C$

$EMPLOYEE *_{Ssn, Essn} DEPENDENT$

$R *_{(A,B), (C,D)} S$ if the join condition is $R.A=S.C$ AND $R.B=S.D$

$EMPLOYEE *_{(Ssn, Dno), (Mgr_ssn, Dnumber)} DEPARTMENT$

38

Complete Set of Relational Algebra Operations

- The set of operations including SELECT (σ), PROJECT (π), UNION ($\cup$), DIFFERENCE ($-$), CARTESIAN PRODUCT ($\times$) and RENAME (ρ) is called *a complete set* because any other relational algebra operation can be expressed by a combination of these operations.
- For example:
 - $R \cap S = ((R \cup S) - (R - S)) - (S - R)$
 $\quad = R - (R - S)$
 - $R \bowtie_{\langle \text{join condition} \rangle} S = \sigma_{\langle \text{join condition} \rangle} (R \times S)$

39

DIVISION Operation

- The division operation is applied to two relations, and denoted by $\div$.
- $Q(Y) \leftarrow R(Z) \div S(X)$, where X should be a proper subset of Z .
 - $Z \supset X$ and $Y = Z - X$ (and hence $Z = X \cup Y$); that is, Y is the set of attributes of R that are not attributes of S .
 - The result of DIVISION is a relation $Q(Y)$ that includes every subtuple $t[Y]$ (containing the values for Y) that appears in R in combination with every tuple $t'[X]$ in S .
 - In other words, for a tuple $t_Q[Y]$ to appear in the result relation Q , $t_Q[Y]$ must appear in R as a subtuple (containing the values for Y) in combination with every tuple in S (containing the values for X).
- The DIVISION can be performed as a sequence of other relational algebra operations:
 $Q(Y) \leftarrow R(Z) \div S(X) \equiv \pi_Y (R) - \pi_Y ((\pi_Y (R) \times S) - R)$

40

Examples of DIVISION

$SSNS(Ssn) \leftarrow SSN_PNOS \div SMITH_PNOS$

$T \leftarrow R \div S$

(a)		(b)	
SSN_PNOS		R	S
Essn	Pno	A	A
123456789	1	a1	a1
123456789	2	a2	a2
666884444	3	a3	a3
453453453	1	a4	b1
453453453	2	a1	b2
333445555	2	a3	b2
333445555	3	a2	b3
333445555	10	a3	b3
333445555	20	a4	b3
999887777	30	a1	b4
999887777	10	a2	b4
987987987	10	a3	b4
987987987	30		
987654321	30		
987654321	20		
888665555	20		

41

Recap of Relational Algebra Operations

Table 8.1 Operations of Relational Algebra

OPERATION	PURPOSE	NOTATION
SELECT	Selects all tuples that satisfy the selection condition from a relation R .	$\sigma_{\langle \text{selection condition} \rangle}(R)$
PROJECT	Produces a new relation with only some of the attributes of R , and removes duplicate tuples.	$\pi_{\langle \text{attribute list} \rangle}(R)$
THETA JOIN	Produces all combinations of tuples from R_1 and R_2 that satisfy the join condition.	$R_1 \bowtie_{\langle \text{join condition} \rangle} R_2$
EQUIJOIN	Produces all the combinations of tuples from R_1 and R_2 that satisfy a join condition with only equality comparisons.	$R_1 \bowtie_{\langle \text{join condition} \rangle} R_2$, OR $R_1 \bowtie_{\langle \text{join attributes 1} \rangle} R_2$ $(\langle \text{join attributes 2} \rangle) R_2$
NATURAL JOIN	Same as EQUIJOIN except that the join attributes of R_2 are not included in the resulting relation; if the join attributes have the same names, they do not have to be specified at all.	$R_1 \star_{\langle \text{join condition} \rangle} R_2$, OR $R_1 \star_{\langle \text{join attributes 1} \rangle} R_2$, $(\langle \text{join attributes 2} \rangle)$ R_2 OR $R_1 \star R_2$

42

Recap of Relational Algebra Operations (cont'd)

Table 8.1 Operations of Relational Algebra

OPERATION	PURPOSE	NOTATION
UNION	Produces a relation that includes all the tuples in R_1 or R_2 or both R_1 and R_2 ; R_1 and R_2 must be union compatible.	$R_1 \cup R_2$
INTERSECTION	Produces a relation that includes all the tuples in both R_1 and R_2 ; R_1 and R_2 must be union compatible.	$R_1 \cap R_2$
DIFFERENCE	Produces a relation that includes all the tuples in R_1 that are not in R_2 ; R_1 and R_2 must be union compatible.	$R_1 - R_2$
CARTESIAN PRODUCT	Produces a relation that has the attributes of R_1 and R_2 and includes as tuples all possible combinations of tuples from R_1 and R_2 .	$R_1 \times R_2$
DIVISION	Produces a relation $R(X)$ that includes all tuples $t[X]$ in $R_1(Z)$ that appear in R_1 in combination with every tuple from $R_2(Y)$, where $Z = X \cup Y$.	$R_1(Z) \div R_2(Y)$

43

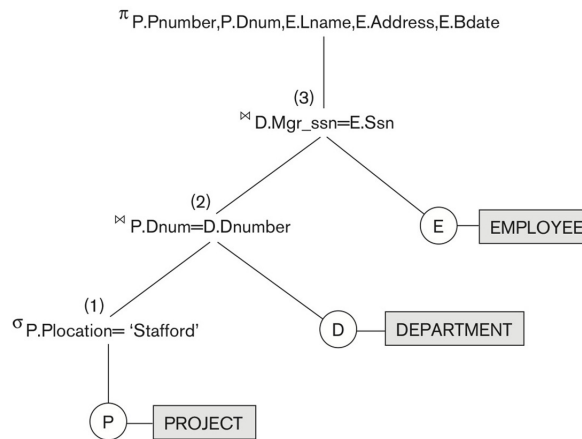
Query Tree

- A relational algebra expression can be represented as a Query Tree, in which
 - Relations (in the database) are represented as leaf nodes, and
 - Relational algebra operations are represented as internal nodes.
- Actually, each SQL query submitted can be parsed and translated into a query tree, and a sequence of relational algebra operations can be obtained from the query tree. (refer to Chapter 19 on Query Optimization)

44

Query Tree Example

$\pi_{Pnumber, Dnum, Lname, Address, Bdate} (((\sigma_{Plocation='Stafford'} (PROJECT)) \bowtie_{Dnum=Dnumber} (DEPARTMENT)) \bowtie_{Mgr_ssn=Ssn} (EMPLOYEE))$



45

Additional Relational Operations: Aggregate Functions and Grouping

- In addition to the basic relational algebra operations, we have mathematical **aggregate functions** each of which can be applied on an attribute (i.e., column) in a relation.
 - For example, retrieving the average salary of all employees or the total number of their SSN values (which is the same as the total number of employees).
- Common aggregate functions applied to a collection of numeric values include
 - **SUM, AVERAGE, MAXIMUM** and **MINIMUM**
- The **COUNT** function is used to count the non-Null values of an attribute in a relation.
- The result of aggregate function(s) on a relation is a relation, even though it may contain only one value.

46

Aggregate Functions

- Use of the Aggregate Function denoted by $\mathcal{F}$ (i.e., 'F' in calligraphic font).
 - $\mathcal{F}_{\text{MAXIMUM Salary}}(\text{EMPLOYEE})$ retrieves the maximum Salary value from the EMPLOYEE relation. The result relation contains a single attribute whose default name is **MAXIMUM_Salary**.
 - $\mathcal{F}_{\text{MINIMUM Salary}}(\text{EMPLOYEE})$ retrieves the minimum Salary value from the EMPLOYEE relation. The result relation contains a single attribute whose default name is **MINIMUM_Salary**.
 - $\mathcal{F}_{\text{SUM Salary}}(\text{EMPLOYEE})$ retrieves the sum of the Salary values from the EMPLOYEE relation. The result relation contains a single attribute whose default name is **SUM_Salary**.
 - $\mathcal{F}_{\text{COUNT Ssn, AVERAGE Salary}}(\text{EMPLOYEE})$ counts the number of SSNs of employees and computes the average Salary value.
 - Note: COUNT just counts the number of non-Null attribute values without removing duplicates. In other words, it counts the number of tuples containing non-Null values in the column of the specified attribute.
 - The result relation contains two attributes whose default names are **COUNT_Ssn** and **AVERAGE_Salary**.

47

Grouping of Tuples and Aggregation

- Grouping can be combined with Aggregate Functions: **group the tuples first, then aggregate functions are applied to each group.**
- Example query: For each department, retrieve the department id, the number of employees in the department and their average salary value.
 - Grouping attributes are placed to the left of symbol $\mathcal{F}$, and Aggregate functions are placed to the right of symbol $\mathcal{F}$:

$$\text{Dno } \mathcal{F}_{\text{COUNT Ssn, AVERAGE Salary}}(\text{EMPLOYEE})$$
 - Above operation groups EMPLOYEE tuples by Dno (i.e., department number) and counts the EMPLOYEE tuples (by counting the SSN values) and computes the average Salary value for each group.
 - **We may have more than one grouping attribute.**

$\text{Dno, Super_ssn } \mathcal{F}_{\text{COUNT Ssn, AVERAGE Salary}}(\text{EMPLOYEE})$

48

Examples of applying aggregate functions and grouping

Figure 8.10 The aggregate function operation.

- (a) $\rho_R(\text{Dno}, \text{No_of_employees}, \text{Average_sal}) (\text{Dno} \bowtie \text{COUNT Ssn}, \text{AVERAGE Salary} (\text{EMPLOYEE}))$
 (b) $\text{Dno} \bowtie \text{COUNT Ssn}, \text{AVERAGE Salary} (\text{EMPLOYEE})$.
 (c) $\bowtie \text{COUNT Ssn}, \text{AVERAGE Salary} (\text{EMPLOYEE})$.

R

(a)

Dno	No_of_employees	Average_sal
5	4	33250
4	3	31000
1	1	55000

(b)

Dno	Count_ssn	Average_salary
5	4	33250
4	3	31000
1	1	55000

(c)

Count_ssn	Average_salary
8	35125

- (a) $R(\text{Dno}, \text{No_of_employee}, \text{Average_sal}) \leftarrow \text{Dno} \bowtie \mathcal{F}_{\text{COUNT Ssn}, \text{AVERAGE Salary}} (\text{EMPLOYEE})$

49

Additional Relational Operations: OUTER JOIN Operations

- The OUTER JOIN Operation
 - In JOIN, tuples (in one operand relation) without a *matching* (or *related*) tuple (in other operand relation) are not included in the join result.
 - Tuples having NULL values for the join attribute are also eliminated.
 - A set of operations, called OUTER JOIN operations, can be used when we want to keep all the tuples in R, or all those in S, or all those in both relations in the result of the join, regardless of whether or not they have matching tuples in the other relation.

50

Outer JOIN Operations

- The **LEFT OUTER JOIN** operation is denoted by $\bowtie\!\!\!\!\!\leftarrow$ and keeps every tuple in the first (or left) relation R in the result of $R \bowtie\!\!\!\!\!\leftarrow S$.
 - For a R tuple, if no matching tuple is found in S, then the R tuple is filled (i.e., “padded”) with NULL values for the attributes of S in the join result.
- The **RIGHT OUTER JOIN** is denoted by $\bowtie\!\!\!\!\!\rightarrow$ and keeps every tuple in the second (or right) relation S in the result of $R \bowtie\!\!\!\!\!\rightarrow S$.
- The **Full OUTER JOIN** is denoted by $\bowtie\!\!\!\!\!\leftrightarrow$ and keeps all tuples in both the left and right relations. When no matching tuple is found for a tuple, it is padded with NULL values as needed.

51

Examples of OUTER JOIN

<u>R (A, B)</u>	<u>S (C, D)</u>
a 10	c 20
b 15	f 5
c 5	a 20

$R \bowtie\!\!\!\!\!\leftarrow S$
A=C

A	B	C	D
a	10	a	20
b	15	Null	Null
c	5	c	20

$R \bowtie\!\!\!\!\!\rightarrow S$
A=C

A	B	C	D
c	5	c	20
Null	Null	f	5
a	10	a	20

$R \bowtie\!\!\!\!\!\leftrightarrow S$
A=C

A	B	C	D
a	10	a	20
b	15	Null	Null
c	5	c	20
Null	Null	f	5

52

Example of LEFT OUTER JOIN

RESULT $\leftarrow \pi_{Fname, Minit, Lname, Dname} (EMPLOYEE \bowtie_{Ssn = Mgr_ssn} DEPARTMENT)$

RESULT

Fname	Minit	Lname	Dname
John	B	Smith	NULL
Franklin	T	Wong	Research
Alicia	J	Zelaya	NULL
Jennifer	S	Wallace	Administration
Ramesh	K	Narayan	NULL
Joyce	A	English	NULL
Ahmad	V	Jabbar	NULL
James	E	Borg	Headquarters

Figure 8.12 The result of a LEFT OUTER JOIN operation.

53

OUTER UNION Operation

- The outer union operation was developed to take the union of tuples from two relations when the relations are *not type compatible* (aka UNION compatible).
- This operation will take the union of tuples in two relations that are **partially compatible**, like $R(X, Y)$ and $S(X, Z)$, meaning that only some of their attributes are type compatible.
 - The attributes that are type compatible are included only once in the result relation, and those attributes that are not type compatible from either relation are also kept in the result relation, like $T(X, Y, Z)$ for the OUTER UNION between $R(X, Y)$ and $S(X, Z)$.

54

Outer UNION Operation (cont'd)

<u>R (X, Y)</u>	<u>S (X, Z)</u>
a 10	a 15
d 6	e 9
c 5	c 7

$T \leftarrow R \text{ OUTER UNION } S$

<u>T (X, Y, Z)</u>
a 10 15
d 6 Null
e Null 9
c 5 7

55

OUTER UNION Operation (cont'd)

- Example: An outer union can be applied to two relations whose schemas are **STUDENT**(Name, Ssn, Department, **Advisor**) and **INSTRUCTOR**(Name, Ssn, Department, **Rank**).
 - The result relation, say **STUDENT_OR_INSTRUCTOR**, will have the following attributes: (Name, Ssn, Department, **Advisor**, **Rank**).
 - Tuples from the two relations are matched based on whether they have the same values for the shared (i.e., common) attributes — Name, Ssn, and Department.
 - If a student is also an instructor, the corresponding single tuple will have values for both **Advisor** and **Rank** in the result relation; otherwise, a student tuple will have NULL for **Rank**, and a faculty tuple will have NULL for **Advisor** in the result relation.

56

Examples of Queries in Relational Algebra – in Multiple Expressions

- **Q1: Retrieve the name and address of all employees who work for the ‘Research’ department.**

$\text{RESEARCH_DEPT} \leftarrow \sigma_{\text{Dname}='Research'}(\text{DEPARTMENT})$
 $\text{RESEARCH_EMPS} \leftarrow (\text{RESEARCH_DEPT} \bowtie_{\text{Dnumber}=\text{Dno}} \text{EMPLOYEE})$
 $\text{RESULT} \leftarrow \pi_{\text{Fname}, \text{Lname}, \text{Address}}(\text{RESEARCH_EMPS})$

- **Q6: Retrieve the names of employees who have no dependents.**

$\text{ALL_EMPS} \leftarrow \pi_{\text{Ssn}}(\text{EMPLOYEE})$
 $\text{EMPS_WITH_DEPS}(\text{SSN}) \leftarrow \pi_{\text{Essn}}(\text{DEPENDENT})$
 $\text{EMPS_WITHOUT_DEPS} \leftarrow (\text{ALL_EMPS} - \text{EMPS_WITH_DEPS})$
 $\text{RESULT} \leftarrow \pi_{\text{Lname}, \text{Fname}}(\text{EMPS_WITHOUT_DEPS} \bowtie_{\text{Ssn}=\text{Ssn}} \text{EMPLOYEE})$

57

Examples of Queries in Relational Algebra – in Single expressions

As a single expression, these queries become:

- **Q1: Retrieve the name and address of all employees who work for the ‘Research’ department.**

$\pi_{\text{Fname}, \text{Lname}, \text{Address}}(\sigma_{\text{Dname}='Research'}(\text{DEPARTMENT}) \bowtie_{\text{Dnumber}=\text{Dno}} (\text{EMPLOYEE}))$

- **Q6: Retrieve the names of employees who have no dependents.**

$\pi_{\text{Lname}, \text{Fname}}((\pi_{\text{Ssn}}(\text{EMPLOYEE}) - \rho_{\text{Ssn}}(\pi_{\text{Essn}}(\text{DEPENDENT}))) \bowtie_{\text{Ssn}=\text{Ssn}} \text{EMPLOYEE})$

58